

MINISTERUL EDUCAȚIEI NAȚIONALE



UNIVERSITATEA TEHNICĂ

DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

Instagram

PROIECT DE SEMESTRU

Student: Mihnea-Daniel Popescu

Student: Claudiu Titiu

Student: Denis Zimbru

Disciplina: Proiectare Software

Cluj-Napoca, 2026

Cuprins

1	Architecture & Tech Stack	2
1.1	Architecture Overview	2
1.2	Backend Technologies	2
1.2.1	Java 21 & Spring Boot	2
1.2.2	Spring Data JPA	3
1.2.3	MySQL & Flyway	3
1.2.4	Lombok	3
1.2.5	Maven & Spring Boot DevTools	3
1.3	Frontend Technologies	3
1.3.1	React JS	3
1.3.2	Multipart File Upload Support	3
1.4	Testing	4
1.4.1	JUnit & Mockito	4
1.4.2	Cypress	4

1 Architecture & Tech Stack

1.1 Architecture Overview

For this project, we chose a classic layered architecture (3-tier architecture) because it keeps the application clean, organized, and easier to maintain as the project grows. Instead of mixing responsibilities together, we separated the application into controllers, services, and repositories. This separation improves readability, simplifies testing, and makes future extensions easier to implement.

The controller layer is responsible for handling HTTP requests and communication with external clients. Controllers remain lightweight and avoid containing business logic directly. The business logic is implemented inside the service layer, which acts as the core of the application. Here, we implemented functionalities such as vote calculations, timestamp management, and entity-to-DTO mapping. The repository layer handles database communication and persistence operations, allowing the rest of the application to remain independent of SQL-specific details.

We also decided to use DTOs (Data Transfer Objects) in order to avoid exposing the internal database structure directly through the API. This approach provides additional flexibility, since backend entities can evolve without affecting API consumers. Another important design decision was the implementation of a self-referencing structure for posts and comments, which enabled support for nested discussions in a scalable and straightforward manner.

1.2 Backend Technologies

1.2.1 Java 21 & Spring Boot

For the backend, we used Java 21 together with Spring Boot, which provided a modern and reliable foundation for the application. Spring Boot simplified the development process through features such as dependency injection, automatic configuration, and integrated web support. These technologies allowed us to focus more on the business logic rather than low-level configuration.

1.2.2 Spring Data JPA

Database interaction was implemented using Spring Data JPA. This framework simplified persistence operations by allowing us to work with Java objects instead of writing SQL queries manually. It also improved code readability and reduced the amount of boilerplate code required for CRUD operations.

1.2.3 MySQL & Flyway

We chose MySQL as the relational database management system because it is reliable, performant, and widely used in real-world applications. To manage database schema changes, we used Flyway. Flyway helped us keep database migrations version-controlled and organized, which is especially important in collaborative development environments.

1.2.4 Lombok

To reduce repetitive boilerplate code, we used Lombok. It automatically generated common methods such as getters, setters, constructors, and builders, keeping the codebase cleaner and easier to maintain.

1.2.5 Maven & Spring Boot DevTools

Maven was used for dependency management and project organization. It allowed us to manage external libraries efficiently and maintain a consistent project structure. Additionally, Spring Boot DevTools improved development speed by enabling automatic application restarts during development.

1.3 Frontend Technologies

1.3.1 React JS

For the frontend, we used React JS to build a modern and interactive user interface. React allowed us to create reusable UI components, manage application state efficiently, and provide a responsive user experience. Its component-based architecture also made the frontend easier to maintain and extend.

1.3.2 Multipart File Upload Support

To support image uploading functionality, we implemented multipart file upload handling using Spring Boot's `MultipartFile` support. This allowed users to upload images efficiently while keeping file management integrated within the backend architecture.

1.4 Testing

1.4.1 JUnit & Mockito

For backend testing, we relied on Spring Boot's testing ecosystem together with JUnit and Mockito. These tools allowed us to verify business logic, validate service behavior, and test repository operations in a structured and reliable manner.

1.4.2 Cypress

For frontend and end-to-end testing, we used Cypress. Cypress allowed us to automate user interaction scenarios and validate the behavior of the application from the user's perspective. This helped ensure that both frontend and backend components worked correctly together.